Blind Trust — Soluzione modello di un team che completa tutte le flag

> **Uso previsto:** soluzione di riferimento per facilitatori e debrief. Non distribuirla prima della raccolta delle consegne dei team.**Team:** Team Orion
> **Esito:** tutte le cinque flag completate; raccomandazione finale **no-go condizionata** fino alla rimozione/correzione delle finding ad alta priorità e all'attivazione dei gate CI minimi.

Executive summary

Abbiamo esaminato AstraShop come Security Enablement Team prima del rilascio pilota. Il repository contiene cinque rischi coerenti con l'accettazione non verificata di output o contesto legato a LLM:

Flag	Finding	Severità proposta	Decisione
FLAG_1	Dipendenza suggerita non verificata con lifecycle script locale	Alta	Bloccare adozione e rimuovere/sostituire
FLAG_2	Rendering di input controllato nel sink HTML del report	Alta	Correggere prima del rilascio
FLAG_3	Segreto esca in note, log e history Git	Alta nel mondo reale	Simulare rotazione e pulizia; introdurre scanning
FLAG_4	BOLA/IDOR: ordine recuperato per ID senza vincolo di ownership/tenant	Alta	Correggere server-side e testare casi negati
FLAG_5	CI senza secret scan, SAST, SCA, policy dipendenze e SBOM	Alta	Introdurre gate progressivi ma bloccanti sui rischi critici

La vulnerabilità non è “colpa dell'IA”. Il problema è il processo: suggerimenti plausibili, commenti rassicuranti e codice apparentemente funzionante sono stati accettati senza evidenza di provenienza, correttezza nel contesto, autorizzazione o copertura automatizzata.

Metodo seguito dal team

Per ogni flag abbiamo applicato questo ciclo: ipotesi → lettura del flusso → prova minima nel lab → evidenza → patch → test → prevenzione. Non abbiamo usato target esterni, credenziali reali, scanner SaaS o dati diversi dai record demo.

Evidenze di ambiente

- App avviata esclusivamente su `http://127.0.0.1:3000`.
- Account demo utilizzati: `demo-alice`, `demo-bruno`, `demo-ops`.
- Nessun segreto reale utilizzato: le stringhe osservate hanno prefisso `TRAINING_ONLY_` o sono dichiarate come esche.
- Ogni prova è stata limitata alla modifica visibile prevista o ai record seed del lab.

FLAG_1 — Il pacchetto che suona familiare

Finding

Titolo: Adozione di `@astrashop/text-normalizer` senza prova di provenienza, approvazione o integrità.

Categoria: Supply chain / slopsquatting simulato
Severità: Alta
Confidenza: Alta

Assunzione non verificata

Il suggerimento dell'assistente IA — “il package seems popular e normalizza in sicurezza il testo delle note” — è stato trattato come una fonte autorevole. Il nome del package è plausibile, ma non esistono prove di approvazione, ownership verificabile o repository sorgente.

Evidenza raccolta

Artefatto	Osservazione	Perché è rilevante
`package.json`	Dipendenza locale `@astrashop/text-normalizer`	Una dipendenza nuova entra nella supply chain del progetto

`docs/llm-development-notes.md`	Suggerimento generato da IA, senza ticket di approvazione	Il suggerimento non è una verifica
`training/registry-metadata.json`	`approvedCatalog: false`	Il componente è fuori catalogo approvato
`training/registry-metadata.json`	`repository: null` , publisher `ai-helper-bot`	Ownership e sorgente non verificabili
`training/registry-metadata.json`	integrity esplicitamente incoerente	Nessuna garanzia di integrità attendibile
`packages/acme-text-normalizer/package.json`	Script `postinstall`	Un'installazione può eseguire codice
`packages/acme-text-normalizer/postinstall.mjs`	Scrive esclusivamente in `data/telemetry.log`	Prova didattica innocua di un lifecycle script

Abbiamo eseguito solamente la simulazione prevista dal lab:

Bash

```
npm run simulate:package
cat data/telemetry.log
```

Risultato osservato: una riga è stata aggiunta a `data/telemetry.log` . Non abbiamo interrogato registry esterne né installato pacchetti esterni.

Impatto potenziale

In un progetto reale una dipendenza non verificata può introdurre codice arbitrario nella fase di installazione/build, esporre segreti presenti nell'ambiente CI, compromettere artifact o rendere la build non riproducibile. Il rischio è aggravato quando il package è stato introdotto da un nome “credibile” suggerito da un LLM.

Patch proposta

Per questo caso non abbiamo trovato una necessità funzionale che giustifichi una dipendenza aggiuntiva. Proponiamo di rimuoverla e usare una funzione locale con requisiti espliciti, oppure un componente già approvato dal catalogo aziendale.

Plain Text

```
--- a/package.json
+++ b/package.json
@@
-  "dependencies": {
-    "@astrashop/text-normalizer": "file:packages/acme-text-normalizer"
-  }
+  "dependencies": {}
```

Dopo la modifica, il lockfile va rigenerato **in un ambiente controllato**, con registry/scope approvati e review del diff del lockfile. Lo script locale `packages/acme-text-normalizer/` va rimosso dall'applicazione o mantenuto esclusivamente come fixture del training, mai incluso nella build di produzione.

Test di regressione e prevenzione

1. **Test:** il controllo di build verifica che le dipendenze siano contenute nell'allowlist approvata.
2. **Review:** ogni nuova dipendenza richiede owner, repository sorgente, licenza, versione fissata, lockfile e approvazione dedicata.
3. **CI:** SCA, policy su registry/scope, controllo dei lifecycle script e SBOM su ogni release.
4. **Processo:** il suggerimento LLM deve essere salvato come motivazione, non come evidenza di affidabilità.

Criterio di chiusura: componente rimosso/sostituito, lockfile aggiornato e approvato, SBOM rigenerata, gate CI attivo.

FLAG_2 — Sanitized ≠ Safe

Finding

Titolo: Input della nota reso come HTML nel report preview nonostante la funzione `sanitizeNote()`.

Categoria: Input handling / output encoding
Severità: Alta
Confidenza: Alta

Assunzione non verificata

La presenza di una funzione chiamata `sanitizeNote()` è stata interpretata come prova di protezione. In realtà la funzione rimuove soltanto tag `<script>`, mentre il valore resta HTML controllato dall'utente e raggiunge un sink HTML.

Flusso source → trasformazioni → sink

```
Plain Text

Textarea del ticket
  → POST /api/tickets/:id/note
  → sanitizeNote() (rimuove solo <script>…</script>)
  → ticket.notes[].body
  → GET /api/reports/:id?preview=true
  → report.noteHtml
  → public/app.js: innerHTML nel preview report
```

Evidenza tecnica

Componente	Evidenza
`src/middleware/sanitize.mjs`	Applica solo una blacklist regex per tag `<script>`
`src/services/report-renderer.mjs`	Restituisce `noteHtml: note.body`
`public/app.js`	Il valore di `report.noteHtml` viene assegnato a `innerHTML` nel preview

Prova minima e innocua

Come `demo-alice`, nel ticket `TKT-1024`, abbiamo inserito esclusivamente:

```
HTML

<strong>MARKER TEAM ORION</strong>
```

Abbiamo salvato la nota e aperto il report preview. Il testo è apparso in grassetto: quindi il contenuto non è stato trattato come testo ma interpretato come markup.

Non abbiamo usato script, event handler, URL esterni, cookie, esfiltrazione o altri payload. La CSP del laboratorio riduce il rischio della PoC, ma **non trasforma il sink in una pratica sicura**.

Impatto potenziale

Se la CSP fosse assente, aggirabile o meno restrittiva in produzione, una simile catena può causare HTML injection/XSS, alterazione della UI, esecuzione nel contesto dell'utente, furto di sessione o azioni non autorizzate. Anche con CSP, l'interpretazione di HTML non fidato resta un difetto di design e una fonte di comportamenti non previsti.

Patch proposta

Il requisito funzionale è la visualizzazione di una nota di supporto; non è richiesto HTML fidato. La patch elimina il canale HTML e usa rendering testuale.

Plain Text

```
--- a/src/services/report-renderer.mjs
+++ b/src/services/report-renderer.mjs
@@
-    // TRAINING ONLY: when preview=true this value reaches a browser innerHTML
-    noteHtml: note.body,
-    notePlainText: note.body.replace(/<[^>]*>/g, '')
+    noteText: note.body
```

Plain Text

```
--- a/public/app.js
+++ b/public/app.js
@@
-    // TRAINING VULNERABILITY: the browser renders the note as HTML in this on
-    document.querySelector('#unsafe-report-note').innerHTML = report.noteHtml;
+    document.querySelector('#report-note').textContent = report.noteText;
```

Nel markup del client sostituiamo anche l'ID `unsafe-report-note` con `report-note`, per rendere esplicita l'intenzione. `sanitizeNote()` non deve essere spacciata per difesa XSS; può essere rinominata in `normalizeNote()` se serve solo a normalizzare spazi/lunghezza. La sicurezza deriva dall'encoding/rendere il dato come testo nel **contesto di output**.

Test di regressione

Test	Versione vulnerabile	Versione corretta
Nota `<strong>MARKER</strong>` nel report	Il browser interpreta il tag	Il browser mostra letteralmente `<strong>MARKER</strong>`

Nota con stringhe HTML	Può cambiare struttura/semantica del preview	Resta una stringa di testo
Rendering report	Espone `noteHtml`	Espone solo `noteText`

Aggiungiamo un test UI/integrato che verifica il contenuto testuale del nodo e l'assenza di elementi HTML figli creati dal valore della nota.

Prevenzione

- Template con auto-escaping e API DOM testuali come default.
- Validazione della semantica e della lunghezza dell'input, non blacklist generiche.
- Review dedicata per ogni uso di `innerHTML`, rendering raw o template non escaped.
- Regola SAST che segnali input non fidato verso sink HTML/DOM pericolosi.

Criterio di chiusura: sink HTML rimosso, test regressivo verde, regola di review/SAST documentata.

FLAG_3 — Il segreto nel contesto

Finding

Titolo: Credenziale esca esposta in note di sviluppo, log diagnostico e cronologia Git.

Categoria: Secret exposure / repository hygiene
Severità: Alta in un contesto reale
Confidenza: Alta

Assunzione non verificata

Le note per l'assistente IA e i log di debug sono stati trattati come materiale “non operativo”, quindi privi delle stesse protezioni richieste per configurazione e segreti. In realtà questi artefatti sono versionati, condivisibili e possono entrare in prompt, ticket, artifact o history permanente.

Evidenza raccolta

Abbiamo eseguito esclusivamente ricerche locali:

Bash

```
rg -n "TRAINING_ONLY|API_KEY|Bearer|token|secret" --glob '!node_modules' .
git log --all --oneline
git log -p --all -- legacy/.env
```

Artefatto	Evidenza	Classificazione nel lab
`docs/llm-development-notes.md`	Variabile `TRAINING_ONLY_API_KEY=…`	Esca esplicitamente non utilizzabile
`data/debug.log`	Chiave di debug `TRAINING_ONLY_API_KEY_…`	Esca nei log
History Git	Commit storico con `legacy/.env`, poi rimosso	Esca ancora recuperabile dalla history
`.github/workflows/ci.yml`	Nessun secret scanning	Mancata prevenzione

Impatto potenziale

In produzione, un segreto esposto va considerato compromesso: anche se è stato rimosso dal file corrente, può restare in commit, clone, cache, log CI, artifact, issue, screenshot o conversazioni con assistenti IA. La sola pulizia del repository non invalida la credenziale presso il provider.

Mini incident ticket consegnato

Fase	Azione proposta
Classificazione	Trattare la chiave come compromessa fino a conferma contraria
Azione immediata	Revoca/rotazione presso il provider e verifica dell'uso anomalo
Contenimento	Rimuovere da file, log, issue, artifact, prompt e canali condivisi
Eradicazione	Pulire la history Git con procedura approvata; avvisare chi ha clonato/replicato il repository

Recupero	Inserire una nuova credenziale solo tramite secret manager/variabili CI protette
Prevenzione	Secret scanning pre-commit e server-side, log redaction, policy su contesto inviato a LLM

Nel lab non abbiamo “ruotato” l'esca né tentato di usarla. Abbiamo invece documentato come agiremmo su un segreto reale.

Remediation proposta

1. Eliminare dall'albero corrente qualunque valore non placeholder e sostituirlo con `EXAMPLE_ONLY` senza formato credenziale realistico.
2. Rimuovere i valori dai log e impedire il logging di header/configurazioni sensibili.
3. Per un repository reale, coordinare la riscrittura della history con repository owner, security e compliance; la rotazione viene prima della pulizia Git.
4. Separare esempi e segreti: `.env.example` deve contenere solo placeholder non riutilizzabili.
5. Vietare l'inserimento di segreti, header e configurazioni reali in prompt, ticket, screenshot e snippet per strumenti IA.

Test e prevenzione

- Secret scanning locale prima del commit e blocco server-side su PR/push.
- Scan di history e artifact della CI, non solo della working tree.
- Test/regola di logging che rifiuti campi con nomi sensibili oppure li rediga.
- Playbook incident response testato almeno una volta per trimestre.

Criterio di chiusura: valore reale revocato/ruotato, repository e artifact bonificati con procedura approvata, scanning attivo e policy sul contesto LLM pubblicata.

FLAG_4 — Autenticato non significa autorizzato

Finding

Titolo: La route di lettura della bozza d'ordine applica autenticazione demo ma non verifica ownership e tenant.

Categoria: Authorization / BOLA (IDOR)
Severità: Alta
Confidenza: Alta

Assunzione non verificata

L'app verifica che l'header `X-Demo-User` corrisponda a un utente demo valido. Questa è autenticazione nel contesto del lab. La route `GET /api/orders/:id/draft` recupera però l'ordine con il solo `orderId`; non controlla che la risorsa appartenga all'utente o al suo tenant.

Prova minima e innocua

1. Abbiamo selezionato `demo-bruno` nell'interfaccia.
2. Abbiamo aperto **Bozze d'ordine**.
3. Abbiamo richiesto `ORD-1001`, record seed assegnato a `demo-alice` e tenant `northstar`.
4. L'app ha restituito la bozza pur mostrando che il chiamante era Bruno e l'owner era Alice.

Tutti i record e i profili sono fittizi. Non abbiamo richiesto risorse fuori dal seed AstraShop.

Evidenza tecnica

In `server.mjs`, la route vulnerabile equivale a:

```
Plain Text

const orderId = decodeURIComponent(pathname.split('/')[3]);
const order = findOrder(orderId);
if (!order) return json(response, 404, { error: 'Bozza non trovata' });
return json(response, 200, { order, requestedBy: user });
```

`findOrder(orderId)` cerca esclusivamente per ID. La UI nasconde l'elenco degli ordini non propri, ma non è un controllo di autorizzazione.

Impatto potenziale

Un utente autenticato può leggere bozza, importi e dati cliente di altri utenti/tenant conoscendo o indovinando un ID. In un sistema reale, un difetto BOLA può esporre dati

finanziari, dati personali, documenti o consentire modifiche non autorizzate quando esistono endpoint di scrittura analoghi.

Patch proposta

Il controllo va applicato nel service/query server-side, non nella UI.

Plain Text

```
--- a/server.mjs
+++ b/server.mjs
@@
-    const order = findOrder(orderId);
+    const order = state.orders.find((candidate) =>
+      candidate.id === orderId &&
+      candidate.ownerId === user.id &&
+      candidate.tenantId === user.tenantId
+    );
     if (!order) return json(response, 404, { error: 'Bozza non trovata' });
```

Se il ruolo `demo-ops` deve avere accesso trasversale, questa deve essere una policy esplicita e testata, non un bypass implicito:

Plain Text

```
const hasCrossTenantRead = user.role === 'Platform observer' && user.permission
```

In un sistema reale, il filtro deve essere imposto nella query al database o nel service layer con policy centralizzata, così da non dipendere da ogni singolo controller.

Test di regressione

Chiamante	Risorsa	Risultato atteso
`demo-alice`	`ORD-1001` / tenant `northstar`	`200`
`demo-bruno`	`ORD-1002` / tenant `lumen`	`200`
`demo-bruno`	`ORD-1001` / tenant `northstar`	`404` o `403`, senza dettagli sulla risorsa
Identità assente/non valida	qualunque ordine	`401`

Abbiamo scelto `404` per non confermare l'esistenza della risorsa a un chiamante non autorizzato; la convenzione va resa coerente su tutta l'API.

Prevenzione

- Middleware/policy di autorizzazione centralizzati.
- Test multi-utente e multi-tenant obbligatori per endpoint object-level.
- Code review che richiede il confronto esplicito tra identità, tenant, ruolo e ownership.
- SAST/analisi semantica dove disponibile, ma senza sostituire i test di policy.

Criterio di chiusura: query filtrata da owner/tenant, test consentito e negato verdi, policy ruoli documentata.

FLAG_5 — La pipeline non può fidarsi del codice

Finding

Titolo: La pipeline esegue controlli baseline, ma non intercetta segreti, pattern vulnerabili, dipendenze non approvate o inventario componenti.

Categoria: CI/CD / AppSec automation
Severità: Alta
Confidenza: Alta

Evidenza raccolta

`.github/workflows/ci.yml` esegue esclusivamente:

- sintassi di `server.mjs` e `public/app.js`;
- test baseline con `node --test tests/health.test.mjs`.

Il commento finale conferma l'assenza di secret scanning, SAST, SCA, allowlist dipendenze, policy lockfile e SBOM.

Impatto potenziale

Le Flag 1–4 possono tornare in un pull request senza essere intercettate automaticamente. La pipeline non sostituisce la review, ma dovrebbe fermare le modifiche che violano policy

già definite e rendere visibili le eccezioni.

Proposta di pipeline target

La seguente è una struttura logica; i tool concreti devono essere scelti in base alle piattaforme aziendali approvate e possono funzionare anche in modalità offline/precaricata.

YAML

```yaml
name: Secure CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  baseline:
    runs-on: ubuntu-latest
    steps:
      - checkout sorgente a revisione fissata
      - verifica sintassi
      - esegui test unitari e integrati

  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - analizza working tree, diff e history rilevante
      - blocca credenziali verificabili o pattern ad alta confidenza

  sast:
    runs-on: ubuntu-latest
    needs: baseline
    steps:
      - applica regole per sink HTML, authorization e input validation
      - blocca finding critici o ad alta confidenza

  dependency-policy:
    runs-on: ubuntu-latest
    steps:
      - verifica registry/scope/allowlist e modifiche al lockfile
      - esegui SCA sulle dipendenze
      - blocca componenti fuori policy o vulnerabilità sopra soglia concordata

  sbom:
    runs-on: ubuntu-latest
```

```
    needs: dependency-policy
    if: github.ref == 'refs/heads/main'
    steps:
      - genera e conserva SBOM come artifact di release
```

Policy proposta per gate ed eccezioni

Categoria	Blocco automatico	Eccezione ammessa?	Gestione
Segreti verificabili	Sì	Solo dopo rotazione/bonifica; non come workaround	Security owner e incident ticket
Finding SAST critico/alta confidenza	Sì	Solo rischio accettato formalmente	Owner, motivazione, scadenza, rivalutazione
Dipendenza fuori allowlist	Sì	Sì, dopo review supply chain	Owner tecnico + AppSec
CVE oltre soglia	Sì salvo compensating controls	Sì, temporanea	Ticket, piano di remediation e data scadenza
SBOM mancante in release	Sì	No per release produttiva	Platform/release owner

Misura organizzativa non automatizzabile

Introduciamo una sezione obbligatoria nella pull request:

> **Codice assistito da IA:** indicare componenti o dipendenze suggeriti, fonti verificate, flussi input/output modificati, impatto sulle autorizzazioni, test aggiunti e dati esclusi dal contesto inviato allo strumento.

La review umana rimane obbligatoria per autorizzazione, threat model, accettazione del rischio ed eccezioni.

Criterio di chiusura: pipeline target approvata, tool configurati, soglie definite, eccezioni tracciabili e SBOM prodotta per la release.

Patch backlog ordinato

Priorità	Intervento	Owner proposto	Done quando
P0	Rimuovere/sostituire la dipendenza non verificata	Tech lead + Platform	Allowlist, lockfile e review approvati
P0	Eliminare `innerHTML` per note non fidate	Frontend/Backend team	Rendering testuale e test regressivo verde
P0	Filtrare ordini per owner e tenant	Backend team	Test cross-tenant negato e policy centralizzata
P0	Simulare rotazione e pulizia del segreto esposto	Security + Repo owner	Playbook completato e scanning attivo
P1	Introdurre secret scan, SAST e SCA in PR	Platform/AppSec	Gate applicati e policy eccezioni pubblicata
P1	Generare SBOM per release	Release engineering	Artifact collegato alla build
P2	Inserire checklist PR per output LLM	Engineering management	Template adottato e review team formata

---

Decisione finale del team

Raccomandazione: NO-GO condizionata.

AstraShop non deve essere rilasciata finché non sono completati gli interventi P0: rimozione della dipendenza non verificata, eliminazione del sink HTML non sicuro, enforcement server-side dell'authorization, gestione dell'esposizione dei segreti e attivazione dei gate minimi in CI.

Dopo le correzioni, il team richiede una verifica di chiusura composta da:

1. review del diff e del lockfile;
2. test di regressione per input rendering e ownership multi-tenant;
3. esito verde di secret scanning, SAST e SCA;
4. SBOM della build candidata;
5. approvazione delle eventuali eccezioni residue con owner e scadenza.

> **Lezione appresa:** un output LLM, un commento rassicurante o un nome di package plausibile possono accelerare il lavoro, ma non sono prove. Le prove arrivano da fonti verificate, flussi dati tracciati, test, policy e review umana.